arXiv:2408.13482v2 [cs.LG] 3 Sep 2024

MPruner: Optimizing Neural Network Size with CKA-Based Mutual Information Pruning

Seungbeom Hu[1*], ChanJun Park[2*], Andrew Ferraiuolo[3], Sang-Ki Ko[4], Jinwoo Kim[5], Haein Song[2], Jieung Kim[6]

[1]Samsung SDS, South Korea
seungbeom.hu@samsung.com
[2]Department of Computer Engineering, Inha University, Incheon, South Korea
{12181613, 22241401}@inha.edu
[3]Independent Researcher, London, United Kingdom
andrewferr@proton.me
[4]Department of Artificial Intelligence, University of Seoul, Seoul, South Korea
sangkiko@uos.ac.kr
[5]Department of Computer Science and Engineering, Seoul National University, Seoul, South Korea
jinwoo.kim@sf.snu.ac.kr
[6]Department of Computer Science, Yonsei University, Seoul, South Korea
jieungkim@yonsei.ac.kr

Abstract

Determining the optimal size of a neural network is critical, as it directly impacts runtime performance and memory usage. Pruning is a well-established model compression technique that reduces the size of neural networks while mathematically guaranteeing accuracy preservation. However, many recent pruning methods overlook the global contributions of individual model components, making it difficult to ensure that a pruned model meets the desired dataset and performance requirements. To address these challenges, we developed a new pruning algorithm, MPruner, that leverages mutual information through vector similarity. MPruner utilizes layer clustering with the Centered Kernel Alignment (CKA) similarity metric, allowing us to incorporate global information from the neural network for more precise and efficient layer-wise pruning. We evaluated MPruner across various architectures and configurations, demonstrating its versatility and providing practical guidelines. MPruner achieved up to a 50% reduction in parameters and memory usage for CNN and transformer-based models, with minimal to no loss in accuracy.

Introduction

Neural networks have revolutionized numerous fields, including computer vision and natural language processing (Abiodun et al. 2018; Elad, Kawar, and Vaksman 2023; Otter, Medina, and Kalita 2020), offering unprecedented levels of accuracy and functionality. However, with the success of neural networks also has come a significant increase in the size and complexity of neural networks, perhaps more so than strictly required: current neural networks often suffer from over-parametrization, where more neurons are used than necessary to effectively represent the dataset. Deployment of such over-parametrized models leads to unnecessary use of memory and power, which also translates to increased financial costs.

Over-parametrization is a significant issue across all model architectures, and the recent success of transformer-based models (Vaswani et al. 2017; Lin et al. 2022) has magnified this trend. These models have achieved high performance on various tasks (Dosovitskiy et al. 2021; Khan et al. 2022; Shamshad et al. 2023) with extremely large networks, often exceeding billions of parameters. While transformers have opened a new chapter in performance, their size introduces new challenges, such as the need to fine-tune pretrained models with data specific to new goals rather than training new models from scratch (Radiya-Dixit and Wang 2020; Tay et al. 2022; Zaken, Goldberg, and Ravfogel 2022). This trend forces users to maintain unnecessarily large pretrained models (Fan et al. 2024), which limits the applicability of these powerful models, especially in resource-constrained environments like edge computing, where inference occurs on phones or other small devices. Consequently, there is a growing need for innovative solutions that try to find out the proper model size without sacrificing the benefits of advanced models.

One way to address this problem is through pruning (Cheng, Zhang, and Shi 2023; Han, Mao, and Dally 2016; Molchanov et al. 2016; Ma, Fang, and Wang 2023; Frantar and Alistarh 2023; Sun et al. 2024; Ganesh, Corso, and Sekeh 2020; Fan et al. 2021; Yang, Cao, and Zhao 2024; Liu et al. 2024), a well-known technique for compressing neural networks. While there is a substantial body of literature on this topic, most approaches focus on evaluating the importance of individual weights or optimizing an objective function subject to sparsity constraints. However, these methods often face two major challenges: (1) irregular memory access resulting from the selective pruning of weights within the same memory region, and (2) the lack of a multi-layer, cluster-wise approach with mathematical guarantees, due to the complexity and difficulty of analyzing the effects of layer clusters on model functionality. As a result, they

*These authors contributed equally.

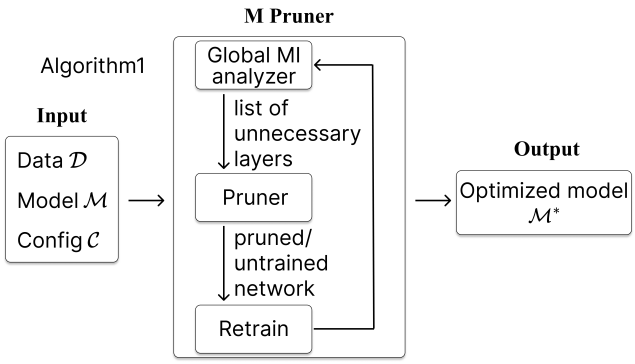

Figure 1: Overview of MPruner

offer a localized best-effort pruning solution, which is effective but fails to provide mathematical proof that the pruned size is optimal for the model and dataset.

To overcome these limitations, we propose a novel pruning algorithm, MPruner, that leverages the Centered Kernel Alignment (CKA) similarity metric (Kornblith et al. 2019) and global multi-layer collapsing, described in Figure 1. MPruner first evaluates the contribution of each layer to the model, removes all unnecessary layers simultaneously, and then retrains the optimized model, resulting in a significantly smaller model size compared to the original. Unlike most previous pruning methods, MPruner uses an accuracy threshold rather than a sparsity level as the pruning criterion. MPruner also determines how many layers can be mathematically and safely pruned while staying within the specified threshold. We validated MPruner by applying it to various architectures, including CNN and transformer-based models, across a wide range of configurations. We also compared and discussed MPruner with earlier pruning methods, including mutual information-based techniques (Ganesh, Corso, and Sekeh 2020; Fan et al. 2021; Guo et al. 2023; Yang, Cao, and Zhao 2024; Ma, Fang, and Wang 2023), and integrated MPruner with other state-of-the-art magnitude pruning methods for transformers (Sun et al. 2024) to explore the potential of combining distinct approaches. Our experiments demonstrate that MPruner is an effective tool for pruning the network to an optimal size for the task with mathematical guarantees. Our experiments also show improvements in efficiency and performance, particularly in memory usage, supported by solid mathematical evidence that we effectively prune unnecessary components. We also provide valuable insights and guidelines for effective application. This highlights the potential of our approach to advance the development and deployment of neural networks, making large and complex models more accessible and sustainable by reducing the required computing resources.

Consequently, our work makes three key contributions, summarized as follows:

- We introduce a novel pruning algorithm, MPruner, which leverages multi-layer cluster-wise information to select pruning candidates. Our approach utilizes comprehensive global information from pre-trained networks, incorporates a retraining phase, and is adaptable to various architectures and pruning configurations.
- We present experimental results demonstrating the effectiveness of our method across various configurations and models. Specifically, we explore four configurations: modularization threshold, accuracy drop threshold, pruning methods, and training methods. Our experiments also cover multiple architectures, including CNNs (ResNet50 and ResNet152), encoder-only transformer architecture (BERT), encoder-decoder transformer architecture (CodeT5), and utilize diverse datasets such as ImageNet, Dair AI, Yahoo! Questions and Answers, and SQuAD.
- We examine the feasibility of integrating our methodology with different structural pruning methods for transformers (Sun et al. 2024) to illustrate how our approach can enhance existing techniques.

Background and Related Works

A large body of work addresses neural network compression, aiming to reduce model size and computational demands while maintaining performance. Among them, three well-known compression methods are quantization, pruning, and knowledge distillation. These methods can be combined rather than chosen as alternatives, as they have orthogonal effects toward the same goal of compression. These techniques are crucial for deploying neural networks in resource-constrained environments, such as mobile devices and edge computing platforms. Our paper focuses on proposing a new pruning method to improve memory and power efficiency.

Pruning

Pruning is broadly categorized into unstructured and structured approaches (Cheng, Zhang, and Shi 2023), each with distinct characteristics and advantages. We focus on structured pruning (He and Xiao 2024), which is extensively studied both for CNNs (Li et al. 2017; Luo, Wu, and Lin 2017; He, Zhang, and Sun 2017; Liu et al. 2017; Lin et al. 2019; Huang and Wang 2018; Ganesh, Corso, and Sekeh 2020) and Transformers (Michel, Levy, and Neubig 2019; Voita et al. 2019; Kovaleva et al. 2019; Lagunas et al. 2021; Ma, Fang, and Wang 2023; Frantar and Alistarh 2023; Sun et al. 2024; Guo et al. 2023; Kim et al. 2024; Ashkboos et al. 2024; Fan et al. 2024; Men et al. 2024; Yang, Cao, and Zhao 2024; An et al. 2024; Fan et al. 2021). This approach simplifies the network's architecture by removing redundant or less critical structures, thereby enhancing overall efficiency. Common techniques in structured pruning include filter pruning, which targets entire filters in convolutional layers to achieve structured sparsity and improve computational efficiency. Neuron pruning, another structural method, involves removing entire neurons in fully connected layers, significantly reducing model size while preserving core functionality. While structured pruning offers the advantage of regular sparsity patterns that are more compatible with optimization on specialized hardware, it often requires careful adjustments to maintain model performance and can

lead to changes in network architecture that impact training dynamics.

Among these approaches, some utilize inter-layer information, such as mutual information within the network, to identify optimal pruning locations. For instance, Mutual Information-based Neuron Trimming (MINT) (Ganesh, Corso, and Sekeh 2020) examines the relationships between filters in consecutive layers of CNN models, removing weights or filters to reduce layer size while considering their relative importance. In the context of transformer models, especially large language models (LLMs), Fan et al. (Fan et al. 2021) determine sparsity levels based on inputs. However, their method focuses primarily on local mutual information by evaluating similarities between adjacent layers and does not consider the contributions of multiple layers simultaneously. LLM-Pruner (Ma, Fang, and Wang 2023) employs structural pruning to selectively remove non-critical coupled structures based on gradient information. SliceGPT (Ashkboos et al. 2024) replaces each weight matrix with a smaller matrix using Principal Component Analysis. LaCo (Yang, Cao, and Zhao 2024) gradually merges similar layers from deep to shallow, setting a threshold to avoid excessive merging of layers. As a result, these methods concentrate on identifying which layers are least or most important and should be pruned or removed, rather than selecting unnecessary layers by analyzing the global multi-layer cluster-wise information of the entire model. Consequently, while they may perform well using local evidence, they cannot demonstrate that the pruned model is optimally sized for the given task.

In this context, MPruner introduces a novel multi-architectural layer pruning approach that significantly enhances the use of analytical metrics by incorporating a state-of-the-art layer similarity metric to guide pruning, overcoming the limitations of previous and related methods. Unlike earlier approaches, MPruner offers two key advantages. First, our method offers both mathematical and intuitive criteria to ensure that pruning safely preserves accuracy. This is achieved by leveraging a well-known metric, Centered Kernel Alignment (CKA) (Kornblith et al. 2019), to calculate layer similarities. Second, it calculates the global contribution of multi-layer clusters rather than focusing solely on pairs of adjacent layers. This allows our method to capture global information more effectively, ensuring a more comprehensive and accurate pruning process.

Centered Kernel Alignment (CKA) similarity score Centered Kernel Alignment (CKA) (Kornblith et al. 2019) is a metric used to assess the similarity between two sets of representations. Equation 5 presents the formula for calculating the CKA value between the outputs of adjacent layers (i and j). It quantitatively measures how well the activations of one layer align with those of another layer or how similar two sets of activations are. Previous research (Davari et al. 2023) has demonstrated that CKA is effective for evaluating the similarity of different layers in neural networks, particularly due to its ability to capture complex and non-linear relationships between activation patterns compared to other methods. As a result, CKA is considered a powerful tool for analyzing and comparing neural network layers.

The process of calculating the CKA score, depicted in Equations 1-5, begins with the outputs of two different layers in the network. For example, if x_i and x_j are the outputs of two adjacent layers, Gram matrices K and L are first computed using Equation 1. These matrices are then centered using a centering matrix (Equations 2 and 3). Next, the Hilbert-Schmidt Independence Criterion (HSIC) (Chwialkowski and Gretton 2014) is calculated using Equation 4, which evaluates the independence between two random variables. Finally, the CKA score between the two layers is obtained using Equation 5. The CKA score offers practical benefits in terms of applicability, as it only requires the output vectors of the networks, eliminating the need to look into the network's detailed structure, such as weight values. In this sense, it can be applied to a wide range of architectures.

$$K = x_i x_i^T \quad L = x_j\, x_j^T \tag{1}$$

$$H = I_n - \frac{1}{n} J_n \quad \text{(Centering Matrix)} \tag{2}$$

$$\hat{K} = HKH \quad \hat{L} = HLH \tag{3}$$

$$HSIC(K, L) = \frac{1}{(n-1)^2}\, tr(\hat{K}H\hat{L}H) \tag{4}$$

$$CKA(x_i, x_j) = \frac{HSIC(K, L)}{\sqrt{HSIC(K, K)\; HSIC(L, L)}} \tag{5}$$

Recent work, DeepArc (Ren et al. 2023), utilizes the CKA metric to demonstrate the potential for clustering multiple adjacent layers into modules within a neural network. They compute the centered Gram matrix for all inputs and then use CKA values to identify layers with similar functions. They also apply DeepArc to ResNet-based models and explore various uses for modularization, including its potential for pruning. However, the work does not provide a concrete algorithm for managing different pruning-related configurations and lacks evidence of broader applicability, including the feasibility of applying their method to transformer models. Additionally, their method for calculating CKA values has room for improvement in both accuracy and scalability. They first store all layer-wise outputs for the given inputs, sum the output values, and then calculate the CKA score. However, this approach can lead to misleading information due to potential overflow when accumulated values are large and when there are significant differences in similarities across individual outputs. Therefore, we develop pruning algorithms, MPruner, that can accommodate multiple configurations, models, and architectures using the CKA value as a core metric, and report our new findings through extensive experimentation.

MPruner: A Pruning Method for Determining Optimal Model Size

MPruner consists of three phases, as depicted in Figure 2. The first phase is the **(1) analysis stage**, where the entire